Tipos Abstractos de Datos (TADs)

Gestión de Catálogo de Productos de Supermercado

1 Contenido

1 Contenido 2
2 TAD: Producto (Entidad Base del Catálogo) 4
2.1 Interfaz Pública 4
2.2 Implementación Privada 4
3 TAD: NodoLista (Nodo de Lista Enlazada) 4
3.1 Interfaz Pública 4
3.2 Implementación Privada 4
4 TAD: ListaSimple (Lista No Ordenada) 5
4.1 Interfaz Pública 5
4.2 Implementación Privada 5
5 TAD: ListaOrdenada (Lista Ordenada Alfabéticamente) 5
5.1 Interfaz Pública 5
5.2 Implementación Privada 5
6 TAD: NodoAVL (Nodo del Árbol AVL) 6
6.1 Interfaz Pública 6
6.2 Implementación Privada 6
7 TAD: ArbolAVL (Árbol Binario de Búsqueda Balanceado) 6
7.1 Interfaz Pública 6
7.2 Implementación Privada 6
8 TAD: NodoB (Nodo del Árbol B) 7
8.1 Interfaz Pública 7
8.2 Implementación Privada 7
9 TAD: ArbolB (Árbol B Orden t=3) 7
9.1 Interfaz Pública 7
9.2 Implementación Privada 8
10 TAD: NodoBPlus (Nodo del Árbol B+) 8
10.1 Interfaz Pública 8
10.2 Implementación Privada 8
11 TAD: ArbolBPlus (Árbol B+ Orden t=3) 9
11.1 Interfaz Pública 9
11.2 Implementación Privada 9
12 TAD: Catalogo (Orquestador Central) 10
12.1 Interfaz Pública 10

12.2 Implementación Privada 10
13 TAD: CargadorCSV (Cargador y Validador del CSV) 11
13.1 Interfaz Pública 11
13.2 Implementación Privada 11
14 TAD: MedidorRendimiento (Benchmark M×N) 11
14.1 Interfaz Pública 11
14.2 Implementación Privada 11
15 TAD: VisualizadorDot (Generador de Archivos Graphviz) 12
15.1 Interfaz Pública 12
15.2 Implementación Privada 12

2 TAD: Producto (Entidad Base del Catálogo)

Representa un producto del supermercado. Es la unidad de datos que almacenan todas las estructuras del sistema. Cada campo actúa como clave de indexación en una estructura distinta.

2.1 Interfaz Pública

```
Producto()
Producto(const string& nombre, const string& codigoBarra,
         const string& categoria, const string& fechaCaducidad,
         const string& marca, double precio, int stock)
void mostrar() const
bool operator<(const Producto& otro) const
bool operator==(const Producto& otro) const
```

2.2 Implementación Privada

Nota: todos los campos son públicos en esta clase para facilitar el acceso directo desde las estructuras.

```
string nombre
string codigoBarra
string categoria
string fechaCaducidad
string marca
double precio
int    stock
```

3 TAD: NodoLista (Nodo de Lista Enlazada)

Unidad estructural compartida por ListaSimple y ListaOrdenada. Almacena un producto y el enlace al siguiente nodo.

3.1 Interfaz Pública

```
explicit NodoLista(const Producto& producto)
```

3.2 Implementación Privada

```
Producto   dato
NodoLista* siguiente
```

4 TAD: ListaSimple (Lista No Ordenada)

Lista enlazada sin orden definido. Inserta siempre al frente (O(1)). Sirve como línea base de comparación en los benchmarks de búsqueda.

4.1 Interfaz Pública

```
ListaSimple()
~ListaSimple()
void      insertar(const Producto& producto)          // O(1)
Producto* buscarPorNombre(const string& nombre)       // O(n)
Producto* buscarPorCodigo(const string& codigo)       // O(n)
bool      eliminar(const string& codigoBarra)         // O(n)
void      listar() const                              // O(n)
int       obtenerTamano() const                       // O(1)
bool      estaVacia() const                           // O(1)
```

4.2 Implementación Privada

```
NodoLista* cabeza
int        tamaño
```

5 TAD: ListaOrdenada (Lista Ordenada Alfabéticamente)

Lista enlazada que mantiene los nodos ordenados por nombre en toda inserción. La búsqueda aplica corte anticipado: si el nodo actual supera al nombre buscado, detiene el recorrido.

5.1 Interfaz Pública

```
ListaOrdenada()
~ListaOrdenada()
void      insertar(const Producto& producto)          // O(n)
Producto* buscarPorNombre(const string& nombre)       // O(n)*
Producto* buscarPorCodigo(const string& codigo)       // O(n)
bool      eliminar(const string& codigoBarra)         // O(n)
void      listar() const                              // O(n)
int       obtenerTamano() const                       // O(1)
bool      estaVacia() const                           // O(1)
```

** Con corte anticipado: termina cuando nombre_actual > buscado.*

5.2 Implementación Privada

```
NodoLista* cabeza
int        tamaño
```

6 TAD: NodoAVL (Nodo del Árbol AVL)

Nodo del árbol AVL. Almacena el producto, los punteros al subárbol izquierdo y derecho, y la altura del subárbol que tiene este nodo como raíz.

6.1 Interfaz Pública

```
explicit NodoAVL(const Producto& producto)
```

6.2 Implementación Privada

```
Producto  dato
NodoAVL*  izquierda
NodoAVL*  derecha
int       altura
```

7 TAD: ArbolAVL (Árbol Binario de Búsqueda Balanceado)

Árbol AVL indexado por nombre del producto. Auto-balanceado mediante rotaciones: la diferencia de altura entre los subárboles izquierdo y derecho de cualquier nodo es siempre ≤ 1, lo que garantiza altura O(log n) y operaciones O(log n).

7.1 Interfaz Pública

```
ArbolAVL()
~ArbolAVL()
void      insertar(const Producto& producto)          // O(log n)
Producto* buscar(const string& nombre) const          // O(log n)
bool      eliminar(const string& nombre)              // O(log n)
void      listarEnOrden() const                       // O(n)
int       obtenerTamano() const                       // O(1)
bool      estaVacio() const                           // O(1)
NodoAVL*  obtenerRaiz() const      // O(1) — para VisualizadorDot
```

7.2 Implementación Privada

```
NodoAVL* raiz
int      tamano
int      obtenerAltura(NodoAVL* nodo) const
int      obtenerBalance(NodoAVL* nodo) const
void     actualizarAltura(NodoAVL* nodo)
NodoAVL* rotarDerecha(NodoAVL* y)
NodoAVL* rotarIzquierda(NodoAVL* x)
NodoAVL* rebalancear(NodoAVL* nodo)
NodoAVL* insertarRec(NodoAVL* nodo, const Producto& producto)
NodoAVL* eliminarRec(NodoAVL* nodo, const string& nombre)
```

```
Producto* buscarRec(NodoAVL* nodo, const string& nombre) const
NodoAVL* obtenerMinimo(NodoAVL* nodo) const
void     listarRec(NodoAVL* nodo) const
void     destruirArbol(NodoAVL* nodo)
```

8 TAD: NodoB (Nodo del Árbol B)

Nodo multi-clave del árbol B. Con orden t=3 puede tener entre 2 y 5 claves (fechas) y entre 3 y 6 hijos. Las claves se almacenan ordenadas de menor a mayor.

8.1 Interfaz Pública

```
NodoB()
```

8.2 Implementación Privada

```
string    claves[MAX_CLAVES]      // MAX_CLAVES = 5  (2t-1, t=3)
Producto  productos[MAX_CLAVES]   // producto asociado a cada clave
NodoB*    hijos[MAX_HIJOS]        // MAX_HIJOS  = 6  (2t,   t=3)
int       numClaves               // claves actualmente almacenadas
bool      esHoja                  // true si no tiene hijos
```

Constantes globales del módulo:

```
static const int ORDEN_B    = 3
static const int MAX_CLAVES = 2 * ORDEN_B - 1  // 5
static const int MAX_HIJOS  = 2 * ORDEN_B      // 6
```

9 TAD: ArbolB (Árbol B Orden t=3)

Árbol B indexado por fecha de caducidad (formato YYYY-MM-DD). Es equivalente a la comparación cronológica. La operación estrella es buscarRango, que recupera todos los productos cuya fecha cae en un intervalo.

9.1 Interfaz Pública

```
ArbolB()
~ArbolB()
void      insertar(const Producto& producto)          // O(log n)
Producto* buscar(const string& fecha) const           // O(log n)
void      buscarRango(const string& fechaInicio,
                      const string& fechaFin) const // O(log n + k)
bool      eliminar(const string& fecha)               // O(log n)
void      listarEnOrden() const                           // O(n)
int       obtenerTamano() const                           // O(1)
bool      estaVacio() const                               // O(1)
```

```
NodoB*    obtenerRaiz() const       // O(1) — para VisualizadorDot
```

9.2 Implementación Privada

```
NodoB* raiz
int    tamano
void     dividirHijo(NodoB* padre, int indice)
void     insertarNoLleno(NodoB* nodo, const string& clave,
                         const Producto& producto)
Producto* buscarEnNodo(NodoB* nodo, const string& clave) const
void     buscarRangoRec(NodoB* nodo, const string& inicio,
                        const string& fin) const
void     eliminarRec(NodoB* nodo, const string& clave)
void     eliminarDeHoja(NodoB* nodo, int idx)
void     eliminarDeInterior(NodoB* nodo, int idx)
void     rellenar(NodoB* nodo, int idx)
void     tomarDePrevio(NodoB* nodo, int idx)
void     tomarDeSiguiente(NodoB* nodo, int idx)
void     fusionar(NodoB* nodo, int idx)
string   obtenerPredecesor(NodoB* nodo, int idx)
string   obtenerSucesor(NodoB* nodo, int idx)
void     listarRec(NodoB* nodo) const
void     destruirArbol(NodoB* nodo)
```

10 TAD: NodoBPlus (Nodo del Árbol B+)

Nodo del árbol B+. En modo hoja almacena productos completos y tiene un puntero siguiente al siguiente nodo hoja (lista enlazada). En modo interno solo almacena claves de navegación; los punteros hijos apuntan a otros nodos.

10.1 Interfaz Pública

```
NodoBPlus()
```

10.2 Implementación Privada

```
string     claves[MAX_CLAVES_BP]      // categorías (claves de nav.)
Producto   productos[MAX_CLAVES_BP]   // solo se usa en hojas
NodoBPlus* hijos[MAX_HIJOS_BP]        // solo se usa en nodos internos
NodoBPlus* siguiente                  // enlace al siguiente nodo hoja
int        numClaves
bool       esHoja
```

Constantes globales del módulo:

```
static const int ORDEN_BPLUS   = 3
```

```
static const int MAX_CLAVES_BP = 2 * ORDEN_BPLUS - 1  // 5
static const int MAX_HIJOS_BP  = 2 * ORDEN_BPLUS      // 6
```

11 TAD: ArbolBPlus (Árbol B+ Orden t=3)

Árbol B+ indexado por categoría del producto. Los datos completos solo viven en las hojas. Las hojas están enlazadas con el puntero siguiente, lo que permite recorrer todos los productos de una categoría en O(k) una vez localizada la primera hoja.

11.1 Interfaz Pública

```
ArbolBPlus()
~ArbolBPlus()
void      insertar(const Producto& producto)        // O(log n)
Producto* buscar(const string& categoria,
                 const string& nombre) const        // O(log n)
void      buscarPorCategoria(const string& categoria)
                              const              // O(log n + k)
bool      eliminar(const string& categoria,
                   const string& codigoBarra)       // O(log n)
void      listarEnOrden() const                         // O(n)
int       obtenerTamano() const                         // O(1)
bool      estaVacio() const                             // O(1)
NodoBPlus* obtenerRaiz() const   // O(1) — para VisualizadorDot
NodoBPlus* obtenerPrimeraHoja() const            // O(1) — para
VisualizadorDot
```

11.2 Implementación Privada

```
NodoBPlus* raiz
NodoBPlus* primeraHoja
int        tamano
void      dividirHijo(NodoBPlus* padre, int indice, NodoBPlus* hijo)
void      insertarNoLleno(NodoBPlus* nodo, const string& clave,
                          const Producto& producto)
Producto* buscarEnNodo(NodoBPlus* nodo, const string& clave) const
void      eliminarRec(NodoBPlus* nodo, const string& clave,
                      const string& codigoBarra)
void      eliminarDeHoja(NodoBPlus* nodo, int idx)
void      rellenarBPlus(NodoBPlus* padre, int idx)
void      tomarDePrevioBP(NodoBPlus* padre, int idx)
void      tomarDeSiguienteBP(NodoBPlus* padre, int idx)
void      fusionarBPlus(NodoBPlus* padre, int idx)
void      destruirArbol(NodoBPlus* nodo)
```

12 TAD: Catalogo (Orquestador Central)

Coordina las cinco estructuras de datos simultáneamente. Garantiza consistencia transaccional: la inserción es atómica (rollback si el AVL detecta nombre duplicado). Nunca se accede a las estructuras directamente desde main.cpp; todo pasa por el Catálogo.

12.1 Interfaz Pública

```
    Catalogo()
    ~Catalogo()
    bool agregarProducto(const Producto& producto)
    bool eliminarProducto(const string& nombre, const string& codigoBarra,
                          const string& categoria, const string&
fechaCaducidad)
    Producto* buscarPorNombre(const string& nombre)
    void      buscarPorCategoria(const string& categoria)
    void      buscarPorRangoFecha(const string& inicio, const string& fin)
    void      listarPorNombre()  const
    void      listarSimple()     const
    void      listarOrdenado()   const
    int       contarProductos()  const
    bool      estaVacio()        const
    ListaSimple*   obtenerListaSimple()   const
    ListaOrdenada* obtenerListaOrdenada() const
    ArbolAVL*      obtenerArbolAVL()      const
    ArbolB*        obtenerArbolB()        const
    ArbolBPlus*    obtenerArbolBPlus()    const
```

12.2 Implementación Privada

```
    ListaSimple*   listaSimple
    ListaOrdenada* listaOrdenada
    ArbolAVL*      arbolAVL
    ArbolB*        arbolB
    ArbolBPlus*    arbolBPlus
    int            totalProductos
    bool existeCodigo(const string& codigoBarra) const
```

13 TAD: CargadorCSV (Cargador y Validador del CSV)

Parsea el archivo CSV y carga los productos al catálogo aplicando validaciones completas. Los registros con error se omiten y se guardan en data/errors.log.

13.1 Interfaz Pública

```
explicit CargadorCSV(const string& rutaLog = "data/errors.log")
int cargar(const string& rutaCSV, Catalogo& catalogo)
```

13.2 Implementación Privada

```
string rutaLog
void   loggearError(int numLinea, const string& motivo,
                    const string& lineaOriginal) const
string limpiarComillas(const string& campo) const
bool   parsearLinea(const string& linea, string campos[],
                    int numCamposEsperados) const
bool   esNumeroValido(const string& texto) const
bool   esEnteroValido(const string& texto) const
```

14 TAD: MedidorRendimiento (Benchmark M×N)

Mide y compara los tiempos de búsqueda por nombre entre ListaSimple, ListaOrdenada y ArbolAVL usando std::chrono::high_resolution_clock. Metodología: M=5 bloques × N=20 búsquedas, promedio en µs.

14.1 Interfaz Pública

```
MedidorRendimiento(ListaSimple* ls, ListaOrdenada* lo, ArbolAVL* avl)
void ejecutar(const string nombresExistentes[], int totalExistentes)
const
```

14.2 Implementación Privada

```
ListaSimple*   listaSimple
ListaOrdenada* listaOrdenada
ArbolAVL*      arbolAVL
static const int N = 20   // búsquedas por bloque
static const int M = 5    // repeticiones del bloque
double medirListaSimple(const string nombres[], int cantidad) const
double medirListaOrdenada(const string nombres[], int cantidad) const
double medirAVL(const string nombres[], int cantidad) const
void   imprimirFila(const string& caso, double usSimple,
                    double usOrdenada, double usAVL) const
void   imprimirEncabezado() const
```

```
    void   imprimirSeparador() const
```

15 TAD: VisualizadorDot (Generador de Archivos Graphviz)

Genera archivos .dot en la carpeta output/ para los tres árboles del sistema.

15.1 Interfaz Pública

```
    explicit VisualizadorDot(const string& carpeta = "output")
    bool generarAVL(ArbolAVL* avl, int maxNodos = 30) const
    bool generarArbolB(ArbolB* arbolB, int maxNodos = 20) const
    bool generarArbolBPlus(ArbolBPlus* arbolBPlus, int maxNodos = 20)
const
    void generarTodos(ArbolAVL* avl, ArbolB* arbolB,
                      ArbolBPlus* arbolBPlus) const
```

15.2 Implementación Privada

```
    string carpetaOutput
    void   avlRec(ofstream& out, NodoAVL* nodo,
                  int& contador, int maxNodos) const
    void   arbolBRec(ofstream& out, NodoB* nodo,
                     int& idNodo, int& contador, int maxNodos) const
    void   arbolBPlusInternosRec(ofstream& out, NodoBPlus* nodo,
                                 int& idNodo, int& contador,
                                 int maxNodos) const
    void   arbolBPlusHojas(ofstream& out, NodoBPlus* primeraHoja,
                           int& idNodo, int maxNodos) const
    string escapar(const string& s) const
```